

Machine Learning II: Explainable AI & Deployment

Week 8, Day 2: Predicting Atherosclerotic Heart Disease

Victor Wandera Lumumba, MSc.

Statistician | Data Analyst | ML Expert

Mediacrest Training College

8th Oct 2026

Abstract

This presentation details the end-to-end development, automation, and deployment of a Machine Learning pipeline to predict Atherosclerotic Heart Disease (AHD).

- **Clinical Objective:** Develop robust predictive models to enable early intervention and risk stratification for cardiac patients.
- **Modeling Strategy:** Train and compare 7 distinct classifiers (including Random Forest and XGBoost) using automated, leakage-free preprocessing pipelines.
- **Evaluation Framework:** Assess models using discrimination (ROC-AUC), class imbalance sensitivity (PR-AUC), and probability calibration.
- **Explainability & Deployment:** Utilize SHAP to demystify "black-box" predictions and deploy the solution via an interactive Streamlit dashboard with 4-tier clinical risk stratification.

Session Objectives

- Formulate a clinical machine learning problem and define the cost of false positives vs. false negatives.
- Construct robust, leakage-free preprocessing pipelines for mixed data types (numerical and categorical).
- Automate the training, evaluation, and serialization of multiple ensemble and baseline models.
- Evaluate model performance using advanced metrics beyond simple accuracy (PR-AUC, Calibration).
- Apply SHAP waterfall plots to explain individual patient predictions and build trust with clinical stakeholders.
- Deploy the final solution as an interactive, medically-themed Streamlit web application.

Part 1: Problem Formulation

"Defining the clinical objective and costs."

Machine Learning & XAI Overview

- **Machine Learning (ML):** Enables computers to learn patterns from data without explicit programming. Branches include Supervised, Unsupervised, Semi-Supervised, and Reinforcement Learning.
- **The "Black Box" Problem:** Complex models like XGBoost offer high accuracy but lack transparency, making clinicians hesitant to trust them.
- **Explainable AI (XAI):** A suite of methods (e.g., SHAP, LIME) designed to make ML predictions transparent, interpretable, and trustworthy.
- **Why XAI in Healthcare?** Clinicians must understand *why* a model flags a patient as high-risk before acting. It fosters trust, enables error detection, and supports shared decision-making.

Problem Formulation: AHD Prediction

Clinical Objective

Predict the presence of Atherosclerotic Heart Disease (AHD) to enable early intervention and risk stratification.

- **Target Variable:** `AHD` (Binary: 0 = No Disease, 1 = Disease Present).
- **Hypotheses:**
  - Ensemble models (RF, XGBoost) will outperform linear baselines due to their ability to capture non-linear interactions.
  - Key predictors will include structural test results (Thal, Ca) over basic vitals.
  - Dynamic class weighting will mitigate bias toward the majority class.

Real-World Clinical Implications

The Cost of Errors

- **False Negatives (Missed Risk):** Delayed treatment, potential acute events (MI, stroke). **HIGH clinical cost.** $\rightarrow$ *Recall/Sensitivity is prioritized.*
- **False Positives (Over-diagnosis):** Unnecessary angiography/stress tests, patient anxiety, healthcare costs. **MODERATE clinical cost.** $\rightarrow$ *Precision balances this.*

- **Calibration:** Clinicians require well-calibrated probabilities (not just raw scores) to make shared decisions with patients regarding invasive procedures.

Part 2: Data Import & Exploration

"Understanding the clinical markers."

Loading Libraries & Dataset

Python Implementation

```
import pandas as pd, numpy as np, shap, xgboost as xgb
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

# Load data with explicit NA handling
df = pd.read_csv('Heart.csv', na_values=['NA', '?', ''])
df['AHD'] = df['AHD'].map({'No': 0, 'Yes': 1}) # Encode target

```

Code Explanation

- We explicitly define missing value indicators (‘NA‘, ‘¿, ‘"""‘) during import to ensure pandas correctly flags them as ‘NaN‘.
- The target variable ‘AHD‘ is mapped to binary integers (0 and 1) for compatibility with scikit-learn classifiers.

Dataset Overview & Key Variables

Variable	Definition & Clinical Relevance	Type
Age	Patient's age in years; strong predictor of CVD risk.	Cont.
ChestPain	Type of pain; asymptomatic indicates severe disease.	Cat.
MaxHR	Max heart rate; lower values indicate poor function.	Cont.
Oldpeak	ST depression; marker of myocardial ischemia.	Cont.
Ca	Major vessels colored; more vessels = severe disease.	Ord.
Thal	Thalassemia test; fixed/reversible defects = ischemia.	Cat.
AHD	Target: Atherosclerotic Heart Disease (0=No, 1=Yes).	Bin.

EDA: Target Distribution & Missing Values

Python Implementation

```
# Target Distribution
print(df['AHD'].value_counts())
# Output: 0 (No): 164, 1 (Yes): 139

# Missing Values
print(df.isnull().sum()[df.isnull().sum() > 0])
# Output: Ca: 4, Thal: 2

```

Graph & Data Explanation

- **Target Balance:** The dataset is relatively balanced (54% No Disease, 46% Disease), which is ideal for training without extreme oversampling.
- **Missingness:** Only 'Ca' (4 missing) and 'Thal' (2 missing) have gaps. These are critical structural markers, so we will impute them carefully in the pipeline rather than dropping rows.

EDA: Numerical & Categorical Features

Graph Explanations (Boxplots & Stacked Bar Charts)

- **Numerical Boxplots:** Reveal that ‘MaxHR‘ is significantly lower in the AHD=1 group, while ‘Oldpeak‘ and ‘Ca‘ are shifted higher. This confirms their strong discriminatory power.
- **Categorical Stacked Bars:** Show the incidence rate of AHD within categories.
  - `ChestPain` type "asymptomatic" shows a massive spike in AHD probability.
  - `Thal` "reversible defect" heavily correlates with positive AHD cases.

Insight

Structural and functional test results (‘Thal‘, ‘Ca‘, ‘ChestPain‘) appear to be stronger drivers of disease presence than general vitals like resting blood pressure.

EDA: Correlation & Multivariate Analysis

Graph Explanations (Heatmap & Pairplot)

- **Correlation Heatmap:** Identifies multicollinearity. High negative correlations between ‘Age‘ and ‘MaxHR‘ are expected physiologically.
- **Multivariate Pairplot:** Visualizes interactions in 2D space.
  - **Ca & Thal Interaction:** Patients with $Ca > 0$ AND a specific ‘Thal‘ defect almost exclusively fall into the ‘AHD=1‘ category.
  - **Age vs MaxHR:** For a given age, AHD patients tend to have an abnormally low MaxHR compared to healthy patients.

Part 3: Preprocessing & Partitioning

"Building robust, leakage-free pipelines."

Data Preprocessing Pipeline

Python Implementation

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OrdinalEncoder(handle_unknown='use_encoded_value',
                               unknown_value=-1))])

preprocessor = ColumnTransformer(
    transformers=[('num', numeric_transformer, num_features),
                  ('cat', categorical_transformer, cat_features)],
    remainder='drop')

```

Code Explanation

We use ‘ColumnTransformer‘ to apply distinct imputation and scaling logic to numerical vs. categorical features, ensuring tree models and distance-based metrics receive clean, encoded data without data leakage.

Data Partitioning Strategy

Python Implementation

```
X = df.drop(columns=['AHD', 'HD']) # Drop redundant target
y = df['AHD']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

```

Code Explanation

- **Stratification:** 'stratify=y' ensures the 54/46 class split is preserved in both train and test sets.
- **Leakage Prevention:** The 'preprocessor' is **not** fitted yet. It will only be fitted inside the cross-validation loops and final model training to prevent test data statistics from leaking.

Verifying Class Proportionality

Class (AHD)	Train (%)	Test (%)
0 (No Disease)	54.1%	54.1%
1 (Disease)	45.9%	45.9%

Table: Class Distribution Across Partitions

Inferential Test (Chi-Square)

- **Chi-square Statistic:** 0.0000
- **P-value:** 1.0000
- **Conclusion:** The p-value > 0.05 confirms there is no statistically significant difference in class proportions between the train and test sets. The stratified split was perfectly executed.

Part 4: Model Training & Automation

"Optimizing and automating the ML pipeline."

Model Selection & Pipeline Setup

Python Implementation

```
models = {
    'Random Forest': RandomForestClassifier(random_state=42, n_jobs=-1),
    'XGBoost': xgb.XGBClassifier(random_state=42, eval_metric='logloss', n_jobs=-1),
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1000),
    'KNN': KNeighborsClassifier(),
    'SVM': SVC(random_state=42, probability=True),
    'Naive Bayes': GaussianNB(),
    'Decision Tree': DecisionTreeClassifier(random_state=42)
}

```

Code Explanation

We define a dictionary of 7 diverse models, ranging from simple baselines (Naive Bayes, Decision Tree) to complex ensembles (Random Forest, XGBoost), allowing for a comprehensive performance comparison.

Automated Training Loop

Python Implementation

```
for name, model in models.items():
    print(f"       Training {name}...")
    pipeline = Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('classifier', model)
    ])

    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)
    y_prob = pipeline.predict_proba(X_test)[:, 1]

    # Calculate metrics and save model
    metrics = {'ROC-AUC': roc_auc_score(y_test, y_prob), ...}
    joblib.dump(pipeline, f'models/{name.replace(" ", "_")}_pipeline.pkl')

```

Automation Benefits

This loop ensures every model undergoes the exact same preprocessing and evaluation protocol, eliminating

Handling Class Imbalance

- Although the dataset is relatively balanced (54% vs 46%), slight imbalances can still bias probability estimates.
- **XGBoost:** Can utilize '$\text{scale}_{p}os_{w}eight'to penalize misclassifications of the minority class more heavily$.
- **Random Forest:** Can handle imbalance via the '$\text{class}_{w}eight =' balanced''hyperparameter, adjusting weights inversely proportional to class frequencies$.
- **Evaluation Focus:** We prioritize **Recall** and **PR-AUC** over simple Accuracy to ensure we do not miss positive cardiac events.

Model Serialization & Persistence

Python Implementation

```
import joblib
import json

# Save the entire pipeline (preprocessor + model)
joblib.dump(pipeline, f'models/{name.replace(" ", "_")}_pipeline.pkl')

# Save metrics for dashboard comparison
with open('models/model_metrics.json', 'w') as f:
    json.dump(metrics_dict, f, indent=4)

```

Code Explanation

- `joblib` is used instead of `pickle` because it is more efficient for objects containing large NumPy arrays (like scikit-learn models).
- Saving the *entire pipeline* ensures the Streamlit app applies the exact same transformations to new patient data as were used during training.

Part 5: Prediction & Evaluation

"Assessing clinical utility and discrimination."

Hold-Out Test Set Evaluation

Python Implementation

```
# Generate predictions and probabilities
y_pred = pipeline.predict(X_test)
y_prob = pipeline.predict_proba(X_test)[:, 1]

# Calculate Precision-Recall AUC (crucial for clinical tasks)
prec, rec, _ = precision_recall_curve(y_test, y_prob)
pr_auc = auc(rec, prec)

```

Code Explanation

- We evaluate on the untouched 20% hold-out set.
- **PR-AUC (Average Precision):** While ROC-AUC evaluates overall ranking, PR-AUC focuses heavily on the model's ability to correctly identify the positive class (Disease) without drowning in false positives. This is the gold standard for medical tasks.

Performance Metrics Comparison (ROC-AUC)

Model	ROC-AUC Score
Random Forest	**0.9589**
Logistic Regression	0.9535
Naive Bayes	0.9524
SVM	0.9470
XGBoost	0.9253
KNN	0.9177
Decision Tree	0.8279

Table: Hold-Out Test Set ROC-AUC Evaluation

Key Takeaway

Random Forest achieves the highest discrimination (0.9589), closely followed by Logistic Regression and Naive Bayes. All ensemble and linear models exceed the 0.94 threshold, indicating excellent clinical utility.

Discriminative Power: ROC & PR Curves

Graph Explanations

- **ROC Curves (AUC $\approx$ 0.96):** Top models hug the top-left corner tightly. This indicates excellent separability; they maintain a high True Positive Rate while keeping the False Positive Rate exceptionally low.
- **Precision-Recall Curves (AP $\approx$ 0.95):** Random Forest performs slightly better here. The RF curve stays closer to the top-right corner (Precision=1, Recall=1) for longer, proving it is more reliable when the cost of false alarms is considered.

Reliability: Calibration & Learning Curves

Graph Explanations

- **Calibration Curves (Reliability Diagrams):**
  - **XGBoost:** Can jump rapidly to 1.0. Very confident, but sometimes slightly miscalibrated in the mid-range.
  - **Random Forest:** Follows a smoother trajectory, tracking closer to the perfect diagonal. It is more "conservative" and reliable for risk stratification.
- **Learning Curves (Bias-Variance):** Training score remains high ($\sim$0.98), while validation score plateaus around 0.89. The visible gap indicates a slight tendency toward overfitting, but generalization remains strong enough for clinical deployment.

Part 6: Feature Importance

"What drives the clinical predictions?"

Feature Importance Analysis (Global Explainability)

Python Implementation

```
Plotting feat¡mp.plot(kind =
'barh', ax=ax, color='007bff')
```

Graph Explanation (Bar Chart)

The horizontal bar chart reveals that structural and functional test features (‘Thal‘, ‘Ca‘, ‘ChestPain‘) dominate predictive importance, accounting for nearly 60% of the model’s decision-making power across tree-based models.

Clinical Insights from Features

- **Thal (Thalassemia Stress Test):** The most significant predictor. A fixed or reversible defect is a massive indicator of ischemia.
- **Ca (Major Vessels Colored):** Direct anatomical indicator of blockage severity. $Ca \geq 2$ almost guarantees a high-risk flag.
- **ChestPain:** "Asymptomatic" pain is counter-intuitively dangerous; it often indicates severe, silent ischemia.
- **Vitals vs. Tests:** Standard metrics like ‘Chol‘ and ‘RestBP‘ have surprisingly low importance. **Insight:** Targeted cardiac diagnostics outweigh general baseline vitals in predicting acute AHD.

Part 7: Explainable AI (SHAP)

"Opening the black box for individual patients."

Introduction to Explainable AI (XAI)

- **The Problem:** A clinician will not order an invasive angiogram based solely on a "black box" outputting ‘0.91‘. They need to know *why*.
- **SHAP (SHapley Additive exPlanations):** Based on cooperative game theory. It assigns each feature an importance value for a *particular* prediction.
- **Waterfall Plots:** Visualize how the model starts at a baseline population risk and adds/subtracts evidence (features) to arrive at the final patient-specific probability.

SHAP Extraction Logic (Robust Handling)

Python Implementation (from app.py)

```
explainer = shap.TreeExplainer(clf)
shap_values = explainer.shap_values(X_test_trans)

# Robust extraction for list and 3D array outputs
if isinstance(shap_values, list):
    shap_values_pos = shap_values[1]
    base_val = explainer.expected_value[1]
elif len(shap_values.shape) == 3:
    shap_values_pos = shap_values[:, :, 1] # Class 1 (Disease)
    base_val = explainer.expected_value[1]
else:
    shap_values_pos = shap_values
    base_val = explainer.expected_value

```

Code Explanation

Different models output SHAP values in different formats. This robust logic ensures the dashboard correctly isolates the positive class (Disease) regardless of whether the model returns a list, a 3D array, or a 2D array.

Interpreting the SHAP Waterfall Plot

Graph Explanation (Waterfall Plot)

- $E[f(x)]$ **(Base Value):** The average predicted probability of AHD across the training set. The model starts with this baseline assumption.
- **Red Bars (Pushing Right):** Features increasing risk. E.g., 'Ca $= 3$' might push probability up by $+0.35$.
- **Blue Bars (Pushing Left):** Features decreasing risk. E.g., 'MaxHR $= 180$' might pull probability down by -0.05.
- $f(x)$ **(Final Output):** The sum of the base value and all SHAP pushes equals the final risk prediction for this specific patient.

Clinical Translation: The Courtroom Analogy

- **The Baseline ($E[f(x)]$):** The starting assumption based on general hospital statistics ("About half the patients we test have heart disease").
- **The Evidence (Red/Blue Bars):** The specific pieces of evidence presented for *this* patient.
  - "The fluoroscopy shows 3 blocked vessels (strong evidence for disease)."
  - "However, their cholesterol is normal (evidence against)."
- **The Verdict ($f(x)$):** The final probability after weighing all evidence. "Based on the heavy evidence of blocked vessels, we are 91.3% certain this patient has the disease."

Part 8: Dashboard Deployment

"Bringing the model to the clinical frontline."

Streamlit Dashboard Architecture

Application Structure (app.py)

The dashboard is built using Streamlit, featuring an 8-page sidebar navigation:

1. **Home:** Overview and best model metrics.
2. **Patient Prediction:** Interactive form for real-time inference.
3. **Model Performance:** Comparative metrics and charts.
4. **Data Visualization:** EDA tools (histograms, heatmaps).
5. **Feature Importance:** Global model interpretability.
6. **Explainability (SHAP):** Local patient-specific waterfall plots.
7. **Dataset Info:** Data dictionary and raw data preview.
8. **Settings:** App configuration and tech stack details.

4-Tier Risk Stratification Logic

Clinical Decision Support

Instead of a binary output, the app provides actionable, tiered recommendations based on the predicted probability:

- **Low Risk ($< 25\%$):** Maintain balanced diet, routine annual check-ups.
- **Moderate Risk (25-50%):** Schedule primary care visit, monitor BP/cholesterol.
- **High Risk (50-75%):** Prompt cardiologist referral, further diagnostics (ECG, stress test).
- **Very High Risk ($\geq 75\%$): Urgent care required.** Immediate cardiology consultation.

Medical Disclaimer

"This tool is for educational and demonstration purposes only. It does not replace professional medical advice, diagnosis, or treatment."

Deployment & Environment Setup

Automation Check (train_model.ipynb)

```
if not os.path.exists('models/model_metrics.json'):
    train_and_save_models()
else:
    print("    Models already exist. Skipping training.")

```

Requirements (requirements.txt)

```
streamlit
pandas
numpy
scikit-learn>=1.0.0
xgboost
shap
matplotlib
seaborn
joblib

```

Final Conclusion & Policy Recommendations

- **Summary:** Ensemble models achieve exceptional diagnostic performance (ROC-AUC > 0.95). Structural features ('Thal', 'Ca') dominate predictive importance over basic vitals.
- **Clinical Deployment:** Deploy the Random Forest pipeline with SHAP explanations to prioritize sensitivity and support shared decision-making via the Streamlit dashboard.
- **Policy Recommendations:**
  - Invest in standardized data capture for high-value features (e.g., fluoroscopy vessel counts).
  - Mandate prospective validation on diverse cohorts before clinical deployment.
  - Integrate interpretable AI literacy into clinical training programs.

Victor Wandera Lumumba
Statistician | Data Analyst | ML Expert
Mediacrest Training College